



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**Semana 1: Título, Problema y Objetivos del Proyecto “Motor de
Integridad Multibase NoSQL”**

Curso: Base de Datos II

Docente: CUADROS QUIROGA, PATRICK JOSE

Integrantes:

Adrian Fabricio Salas Acuña - 2021069829

Keyla Noelia Lopez Ojeda - 2023078690

Tacna – Perú

Índice

Introducción	4
1. Título del Proyecto	5
1.1. Fundamentación del Título	5
2. Planteamiento del Problema	6
2.1. Descripción de la Realidad Problemática	6
2.2. Evidencia del Problema en los Motores Seleccionados	7
2.3. Causas y Efectos del Problema	8
2.4. Formulación del Problema	9
2.4.1. Problema General	9
2.4.2. Problemas Específicos	9
2.5. Justificación	10
2.5.1. Justificación Técnica	10
2.5.2. Justificación Económica	10
2.5.3. Justificación Académica	10
2.6. Delimitación	11
2.6.1. Alcance	11
2.6.2. Fuera del Alcance	11
3. Objetivos	12
3.1. Objetivos de Investigación	12
3.1.1. Objetivo General	12
3.1.2. Objetivos Específicos	12
3.2. Objetivos de la Solución	13
3.3. Matriz de Consistencia	14
Referencias	15

Índice de tablas

Tabla 1 <i>Capacidades Nativas de Integridad en los Motores Seleccionados</i>	8
Tabla 2 <i>Árbol de Problemas: Causas, Problema Central y Efectos</i>	9
Tabla 3 <i>Objetivos Medibles de la Solución</i>	13
Tabla 4 <i>Matriz de Consistencia del Proyecto</i>	14

Introducción

Las aplicaciones actuales rara vez guardan todos sus datos en un único motor. La persistencia políglota, es decir, el uso de distintos almacenes de datos según las necesidades de cada componente de un sistema, se ha consolidado como una práctica habitual en las que conviven bases de datos documentales, de clave-valor y de columnas anchas (Sadalage y Fowler, 2012). De acuerdo con la encuesta para desarrolladores de Stack Overflow (2025), Redis es utilizado por el 28 % de los encuestados y MongoDB por el 24 %, mientras que Apache Cassandra mantiene su presencia en sistemas que requieren escrituras masivas y alta disponibilidad.

Estos motores ganan escalabilidad y flexibilidad a cambio de renunciar a buena parte de las garantías que los sistemas relacionales ofrecen de forma nativa, como las claves foráneas, las restricciones de unicidad y la validación declarativa del esquema (Cattell, 2011; Stonebraker, 2010). El presente informe corresponde a la primera semana del proyecto del curso Base de Datos II y expone el título del proyecto, el planteamiento detallado del problema, sustentado en fuentes académicas y técnicas, y los objetivos de investigación y de la solución, formulados con indicadores medibles.

1. Título del Proyecto

El título propuesto para el proyecto es el siguiente:

Motor de Integridad Multibase NoSQL: Imposición de Reglas de Esquema, Unicidad, Integridad Referencial y Consistencia sobre MongoDB, Redis y Apache Cassandra

1.1. Fundamentación del Título

El título identifica el producto que se construye (un motor de integridad), su alcance (multibase, porque opera de manera simultánea sobre varios motores y entre ellos), el tipo de tecnología intervenida (bases de datos NoSQL) y los cuatro tipos de reglas que el sistema impone: esquema, unicidad, integridad referencial y consistencia. Los motores seleccionados representan los tres modelos de datos NoSQL más extendidos: el documental (MongoDB), el de clave-valor (Redis) y el de columnas anchas (Apache Cassandra), según la clasificación propuesta por Davoudian et al. (2018).

El producto resultante es una aplicación web desplegada en la nube que permite a cada usuario registrar las conexiones a sus propias bases de datos, definir reglas de integridad sin escribir código, validar registros antes de escribirlos, verificar por lotes los datos ya almacenados y auditar las violaciones detectadas desde un tablero y mediante reportes descargables.

Palabras clave: integridad de datos, NoSQL, persistencia políglota, integridad referencial, calidad de datos, MongoDB, Redis, Apache Cassandra.

2. Planteamiento del Problema

2.1. Descripción de la Realidad Problemática

En el modelo relacional, la integridad de los datos es responsabilidad del propio sistema gestor: las claves primarias garantizan la unicidad, las claves foráneas aseguran que toda referencia apunte a un registro existente y los tipos y restricciones del esquema impiden almacenar valores mal formados (Codd, 1970). Las bases de datos NoSQL surgieron precisamente para relajar ese conjunto de garantías y obtener a cambio escalabilidad horizontal, baja latencia y flexibilidad de esquema (Cattell, 2011; Gessert et al., 2017).

Este intercambio tiene un fundamento teórico. El teorema CAP establece que un sistema distribuido sometido a particiones de red no puede ofrecer al mismo tiempo consistencia y disponibilidad (Gilbert y Lynch, 2002), y gran parte de los motores NoSQL privilegian la disponibilidad mediante un modelo de consistencia eventual (Brewer, 2012; Vogels, 2009). En la práctica, el motor acepta escrituras que, desde el punto de vista del negocio, pueden ser incorrectas, y deja en manos de la aplicación la tarea de detectarlas y corregirlas (Kleppmann, 2017).

El problema se agrava cuando una organización combina varios motores. En un escenario típico de comercio electrónico, los usuarios pueden residir en MongoDB, los pedidos en caché en Redis y el historial de pagos en Cassandra. Ningún motor conoce los datos de los demás, por lo que nada impide que un pago registrado en Cassandra haga referencia a un pedido inexistente en MongoDB, ni que un mismo dato replicado en dos motores diverja con el tiempo (Sadalage y Fowler, 2012). Las reglas de integridad terminan dispersas en el código de cada servicio, se duplican, se aplican de forma desigual y no dejan un rastro auditable de las violaciones.

A ello se suma que la flexibilidad de esquema favorece la aparición de registros estructuralmente anómalos: campos faltantes, tipos de dato distintos para un mismo atributo o varias versiones de esquema que conviven en una misma colección (Scherzinger et al., 2013). Klettke et al. (2015) comprobaron, sobre conjuntos de datos reales, que estas

desviaciones estructurales son frecuentes y que su detección exige analizar los datos desde fuera del motor que los almacena.

Las consecuencias afectan directamente la calidad de los datos y, por extensión, la economía de las organizaciones. La exactitud, la completitud y la consistencia son dimensiones centrales de la calidad de datos percibida por sus usuarios (Wang y Strong, 1996), y su aseguramiento requiere metodologías y mediciones explícitas (Batini et al., 2009). Gartner estima que la mala calidad de los datos cuesta a las organizaciones un promedio de 12,9 millones de dólares al año (Sakpal, 2021), y Redman (2016) reporta estimaciones según las cuales los datos de mala calidad le cuestan a la economía de los Estados Unidos alrededor de 3,1 billones de dólares anuales.

2.2. Evidencia del Problema en los Motores Seleccionados

La documentación oficial de los tres motores confirma la ausencia de mecanismos nativos para hacer cumplir las cuatro reglas de integridad, en especial cuando una regla involucra a más de un motor. MongoDB utiliza un modelo de esquema flexible en el que, por defecto, los documentos de una colección no necesitan compartir los mismos campos ni tipos de dato; la validación de esquema es opcional y su nivel de aplicación determina cómo se tratan los documentos ya existentes (MongoDB, Inc., s.f.). Apache Cassandra no admite uniones (*joins*) y declara de forma explícita que no posee el concepto de claves foráneas ni de integridad relacional; los datos se duplican deliberadamente entre tablas mediante la desnormalización (The Apache Software Foundation, s.f.). Redis, por su parte, se define como un servidor de estructuras de datos que almacena valores bajo claves sin un esquema declarado (Redis Ltd., s.f.). La Tabla 1 resume esta situación.

Tabla 1

Capacidades Nativas de Integridad en los Motores Seleccionados

Tipo de regla	MongoDB	Redis	Apache Cassandra
Esquema	Validación opcional con \$jsonSchema por colección; no revisa los documentos ya almacenados	No existe; los valores son cadenas, <i>hashes</i> u otras estructuras sin esquema	Tipos por columna, sin campos obligatorios fuera de la clave primaria ni restricciones de dominio
Unicidad	Índices únicos dentro de una sola colección	Solo la unicidad del nombre de la clave	Solo la clave primaria; insertar una clave repetida sobrescribe el registro
Integridad referencial	No existen claves foráneas	No existe	No existe; no hay claves foráneas ni uniones
Consistencia entre motores	No existe	No existe	No existe

Nota. Elaboración propia a partir de MongoDB, Inc. (s.f.), Redis Ltd. (s.f.) y The Apache

Software Foundation (s.f.).

2.3. Causas y Efectos del Problema

A partir de la revisión anterior se identificaron las causas principales del problema central y los efectos que produce. La Tabla 2 los organiza a modo de árbol de problemas.

Tabla 2

Árbol de Problemas: Causas, Problema Central y Efectos

Nivel	Código	Descripción
Efectos	E1	Datos huérfanos (referencias a registros inexistentes) que provocan errores en procesos y reportes.
	E2	Registros duplicados que distorsionan métricas, inventarios y facturación.
	E3	Datos replicados que divergen entre motores y originan decisiones basadas en información contradictoria.
	E4	Ausencia de trazabilidad: las violaciones no se registran ni pueden auditarse.
	E5	Mayor costo de mantenimiento y de corrección manual de los datos.
Problema central	P	Las organizaciones que usan varios motores NoSQL no cuentan con un mecanismo centralizado para definir, verificar y auditar reglas de integridad sobre sus datos.
Causas	C1	Los motores NoSQL no implementan claves foráneas ni restricciones entre colecciones o entre motores.
	C2	El esquema flexible permite almacenar registros con estructura o tipos incorrectos.
	C3	La consistencia eventual y la desnormalización generan copias de datos que pueden divergir.
	C4	Las reglas de integridad están dispersas y duplicadas en el código de cada aplicación.
	C5	No existen herramientas que validen reglas entre motores heterogéneos de forma independiente de la infraestructura.

Nota. Elaboración propia.

2.4. Formulación del Problema

2.4.1. Problema General

¿De qué manera un motor de integridad multibase puede imponer y verificar reglas de esquema, unicidad, integridad referencial y consistencia sobre datos almacenados en bases de datos NoSQL heterogéneas como MongoDB, Redis y Apache Cassandra?

2.4.2. Problemas Específicos

PE1. ¿Cómo validar la estructura de los registros almacenados en motores NoSQL frente a un esquema declarado, tanto antes de su escritura como sobre los datos ya existentes?

- PE2.** ¿Cómo detectar registros duplicados según uno o varios campos definidos como únicos en motores que no aplican esta restricción de forma nativa?
- PE3.** ¿Cómo verificar que las referencias entre registros de distintos motores apunten a registros existentes?
- PE4.** ¿Cómo detectar divergencias entre los valores de un mismo dato replicado en más de un motor?
- PE5.** ¿Cómo registrar y presentar las violaciones detectadas para que sean trazables y auditables?

2.5. Justificación

2.5.1. Justificación Técnica

El proyecto traslada las reglas de integridad desde el código disperso de cada aplicación hacia un motor centralizado y declarativo. Mediante un adaptador por motor, las mismas reglas se aplican a MongoDB, Redis y Cassandra sin modificar la lógica de validación, y se combinan dos estrategias complementarias: una preventiva, que valida un registro antes de escribirlo, y otra detectiva, que examina por lotes los datos ya almacenados.

2.5.2. Justificación Económica

Dado el costo que la mala calidad de los datos representa para las organizaciones (Redman, 2016; Sakpal, 2021), detectar las violaciones de forma temprana reduce el reprocesamiento y la corrección manual. Además, la solución se construye sobre servicios en la nube con planes gratuitos (MongoDB Atlas, Upstash, DataStax Astra, Supabase y Streamlit Community Cloud), por lo que su costo de operación para fines académicos y de pequeña escala es mínimo.

2.5.3. Justificación Académica

El proyecto aplica los conceptos de integridad, restricciones y modelos de datos estudiados en el curso Base de Datos II y los extiende al contexto NoSQL, en el que dichas garantías no existen de forma nativa. Asimismo, deja una herramienta reproducible que

puede emplearse para demostrar y medir problemas de calidad de datos en arquitecturas de persistencia políglota.

2.6. Delimitación

El proyecto se desarrolla durante el semestre académico 2026-II en la Escuela Profesional de Ingeniería de Sistemas de la Universidad Privada de Tacna.

2.6.1. Alcance

El proyecto comprende los siguientes elementos:

- Los motores MongoDB, Redis y Apache Cassandra alojados en servicios en la nube (MongoDB Atlas, Upstash y DataStax Astra).
- Los cuatro tipos de regla: esquema, unicidad, integridad referencial y consistencia.
- La validación de registros antes de su escritura y la verificación por lotes de los datos ya almacenados.
- El registro cifrado de las credenciales de conexión de cada usuario.
- Un plano de control en Supabase (PostgreSQL) para usuarios, reglas y auditoría.
- Una interfaz web desarrollada con Streamlit, con tablero de violaciones y reportes en PDF.

2.6.2. Fuera del Alcance

No forman parte del proyecto los siguientes aspectos:

- Las transacciones distribuidas entre motores, como el protocolo de confirmación en dos fases.
- La corrección automática de los datos que violan una regla.
- La intercepción de escrituras que aplicaciones de terceros realizan directamente sobre los motores.
- El soporte de otros motores NoSQL, como las bases de datos de grafos.

3. Objetivos

Siguiendo el enfoque de la investigación en ciencia del diseño, que distingue entre el conocimiento generado y el artefacto construido para resolver un problema (Hevner et al., 2004), los objetivos se dividen en objetivos de investigación y objetivos de la solución. Cada objetivo incluye una meta cuantificable que permite verificar su cumplimiento.

3.1. Objetivos de Investigación

3.1.1. *Objetivo General*

Desarrollar y evaluar un motor de integridad multibase que permita definir, verificar y auditar reglas de esquema, unicidad, integridad referencial y consistencia sobre datos almacenados en MongoDB, Redis y Apache Cassandra, y que detecte el 100 % de las violaciones presentes en un conjunto de datos de prueba controlado.

3.1.2. *Objetivos Específicos*

- OE1.** Implementar una validación de esquema basada en JSON Schema que rechace, antes de su escritura, el 100 % de los registros que incumplan el esquema declarado y que identifique los registros ya almacenados que lo incumplen.
- OE2.** Implementar una verificación de unicidad que identifique el 100 % de los registros duplicados según los campos definidos como únicos en cada motor.
- OE3.** Implementar una verificación de integridad referencial entre motores que identifique el 100 % de las referencias huérfanas del conjunto de prueba.
- OE4.** Implementar una verificación de consistencia que detecte el 100 % de las divergencias entre los valores de un mismo dato replicado en dos motores.
- OE5.** Registrar el 100 % de las violaciones detectadas en un historial de auditoría centralizado y presentarlas en un tablero y en reportes descargables en formato PDF.

3.2. Objetivos de la Solución

Los objetivos de la solución describen los resultados verificables que debe alcanzar el sistema construido. La Tabla 3 detalla, para cada uno, el indicador con su fórmula de cálculo, la meta y el instrumento con el que se medirá.

Tabla 3

Objetivos Medibles de la Solución

Código	Objetivo	Indicador y fórmula	Meta	Instrumento
OS1	Soportar los cuatro tipos de regla en los tres motores	Combinaciones regla-motor soportadas / combinaciones previstas (4 × 3)	12 de 12 (100 %)	Lista de verificación funcional y pruebas unitarias
OS2	Detectar las violaciones existentes en los datos	Violaciones detectadas / violaciones inyectadas × 100	100 % (7 de 7)	Escaneo por lotes sobre el conjunto de prueba
OS3	No reportar violaciones inexistentes	Violaciones reportadas por error / violaciones reportadas × 100	0 %	Revisión manual de los reportes
OS4	Rechazar registros inválidos antes de escribirlos	Registros inválidos rechazados / registros inválidos enviados × 100	100 %	Casos de prueba del validador
OS5	Asegurar la trazabilidad de las violaciones	Violaciones registradas con regla, motor, fecha y dato afectado / violaciones detectadas × 100	100 %	Historial de auditoría en Supabase y reporte PDF
OS6	Proteger las credenciales	Credenciales de motores cifradas / credenciales almacenadas × 100	100 %; ninguna credencial en texto plano en el repositorio	Revisión de la base de control y del repositorio
OS7	Garantizar la calidad y la disponibilidad del software	Pruebas unitarias aprobadas / pruebas ejecutadas × 100	100 %; aplicación accesible mediante una URL pública	pytest y Streamlit Community Cloud

Nota. Elaboración propia. El conjunto de datos de prueba contiene siete violaciones

inyectadas, distribuidas entre MongoDB, Redis y Apache Cassandra, que cubren los cuatro tipos de regla.

3.3. Matriz de Consistencia

La Tabla 4 relaciona cada problema con el objetivo que lo atiende y con el indicador que permitirá comprobar su logro, lo que asegura la coherencia entre el planteamiento y la solución propuesta.

Tabla 4

Matriz de Consistencia del Proyecto

Problema	Objetivo	Indicador principal
General. ¿De qué manera un motor de integridad multibase puede imponer y verificar las cuatro reglas sobre MongoDB, Redis y Cassandra?	General. Desarrollar y evaluar un motor de integridad multibase que detecte el 100 % de las violaciones del conjunto de prueba.	Tasa de detección de violaciones (OS2)
PE1. Validar la estructura frente a un esquema	OE1. Validación de esquema con JSON Schema	Registros inválidos rechazados (OS4)
PE2. Detectar registros duplicados	OE2. Verificación de unicidad	Duplicados detectados / duplicados inyectados
PE3. Verificar referencias entre motores	OE3. Verificación de integridad referencial	Referencias huérfanas detectadas / inyectadas
PE4. Detectar divergencias entre réplicas	OE4. Verificación de consistencia	Divergencias detectadas / inyectadas
PE5. Registrar y presentar las violaciones	OE5. Auditoría, tablero y reportes PDF	Violaciones trazables (OS5)

Nota. Elaboración propia.

Referencias

- The Apache Software Foundation. (s.f.). *Introduction* [Modelado de datos]. Apache Cassandra Documentation. Recuperado el 3 de octubre de 2026, de <https://cassandra.apache.org/doc/latest/cassandra/developing/data-modeling/intro.html>
- Batini, C., Cappiello, C., Francalanci, C., y Maurino, A. (2009). Methodologies for data quality assessment and improvement. *ACM Computing Surveys*, 41(3), Artículo 16. <https://doi.org/10.1145/1541880.1541883>
- Brewer, E. (2012). CAP twelve years later: How the “rules” have changed. *Computer*, 45(2), 23–29. <https://doi.org/10.1109/MC.2012.37>
- Cattell, R. (2011). Scalable SQL and NoSQL data stores. *ACM SIGMOD Record*, 39(4), 12–27. <https://doi.org/10.1145/1978915.1978919>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Davoudian, A., Chen, L., y Liu, M. (2018). A survey on NoSQL stores. *ACM Computing Surveys*, 51(2), Artículo 40. <https://doi.org/10.1145/3158661>
- Gessert, F., Wingerath, W., Friedrich, S., y Ritter, N. (2017). NoSQL database systems: A survey and decision guidance. *Computer Science – Research and Development*, 32(3–4), 353–365. <https://doi.org/10.1007/s00450-016-0334-3>
- Gilbert, S., y Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59. <https://doi.org/10.1145/564585.564601>
- Hevner, A. R., March, S. T., Park, J., y Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O’Reilly Media.
- Klettke, M., Störl, U., y Scherzinger, S. (2015). Schema extraction and structural outlier detection for JSON-based NoSQL data stores. En *Datenbanksysteme für Business*,

- Technologie und Web (BTW 2015)* (pp. 425–444). Gesellschaft für Informatik.
<https://dl.gi.de/handle/20.500.12116/2420>
- MongoDB, Inc. (s.f.). *Schema validation*. MongoDB Docs. Recuperado el 3 de octubre de 2026, de <https://www.mongodb.com/docs/manual/core/schema-validation/>
- Redis Ltd. (s.f.). *Redis data types*. Redis Docs. Recuperado el 3 de octubre de 2026, de <https://redis.io/docs/latest/develop/data-types/>
- Redman, T. C. (2016, 22 de septiembre). Bad data costs the U.S. \$3 trillion per year. *Harvard Business Review*. <https://hbr.org/2016/09/bad-data-costs-the-u-s-3-trillion-per-year>
- Sadalage, P. J., y Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.
- Sakpal, M. (2021, 14 de julio). *How to improve your data quality*. Gartner.
<https://www.gartner.com/smarterwithgartner/how-to-improve-your-data-quality>
- Scherzinger, S., Klettke, M., y Störl, U. (2013). Managing schema evolution in NoSQL data stores. En *Proceedings of the 14th International Symposium on Database Programming Languages (DBPL 2013)*. arXiv.
<https://doi.org/10.48550/arXiv.1308.0514>
- Stack Overflow. (2025). *2025 Stack Overflow Developer Survey: Technology*.
<https://survey.stackoverflow.co/2025/technology>
- Stonebraker, M. (2010). SQL databases v. NoSQL databases. *Communications of the ACM*, 53(4), 10–11. <https://doi.org/10.1145/1721654.1721659>
- Vogels, W. (2009). Eventually consistent. *Communications of the ACM*, 52(1), 40–44.
<https://doi.org/10.1145/1435417.1435432>
- Wang, R. Y., y Strong, D. M. (1996). Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems*, 12(4), 5–33.
<https://doi.org/10.1080/07421222.1996.11518099>